Принципы и паттерны проектирования

О себе

Алексей Солдатов

- Backend-разработчик
- В hh работаю с середины 2022 года
- В коммерческой разработке с 2013-го
- На Java пишу с 2015-го

Организационная часть

План лекции

- Принципы проектирования ~40 минут
- Перерыв 10 минут
- Паттерны ООП ~60 минут
- Q&A ~10 минут

О наличии вопросов по ходу лекции буду периодически спрашивать.

Вопросы очень желательно задавать голосом. Текстовые, скорее всего, прочитаю только в конце.

Общаемся на «ты».

Содержание лекции

- Принципы проектирования 5
  - Принципы ООП 6
  - SOLID 11
  - Coupling & Cohesion 17
  - Композиция vs. наследование 23
- Паттерны проектирования 30
  - Введение 31
  - Порождающие 35
    - Builder 36
  - Структурные 39
    - Adapter 40
    - Proxy 43
  - Поведенческие 46
    - Strategy 47
- Ссылки и литература 50

Принципы проектирования

Принципы ООП

Инкапсуляция

Сокрытие данных и деталей реализации. Объект как черный ящик.

```
class Printer {
  private PrinterProperties properties;

  public void print(Object content) {
    prepare();
    /* ... */
  }

  public void setProperties(PrinterProperties properties) {
    this.properties = properties;
  }

  private void prepare() {
    /* ... */
  }
}
```

Наследование

Возможность порождать один класс от другого с сохранением всех свойств и методов и добавляя новые.

```
class MusicInstrument {
  void play() { /* ... */ }
}

class Guitar extends MusicInstrument {
  void play() { /* ... */ }
  void tuneStrings() { /* ... */ }
}
```

Полиморфизм

Возможность использовать классы-потомки в контексте, предназначенном для класса-предка.

```
class MusicInstrument {
  void play() { /* ... */ }
}

class Guitar extends MusicInstrument {
  void play() { /* ... */ }
}

void playMusicInstrument(MusicInstrument instrument) {
  instrument.play()
}

MusicInstrument guitar = new Guitar();
playMusicInstrument(guitar);
```

Абстракция

Использование только тех характеристик объекта, которые с достаточной точностью определяют его в данной системе.

На курсах

```
class Student {
  String name;
  String email;
  String github;
  Map<Lesson, Score> scores;
}
```

В университете

```
class Student {
  String name;
  String phone;
  String homeAddress;
  String recordBookId;
  Faculty faculty;
  Integer groupNumber;
}
```

SOLID

Single Responsibility Principle

У класса должна быть только одна причина для изменения.

Пример нарушения

```
class Report {
  String getContent() { /* ... */ }
  void print() { /* ... */ }
}
```

Пример соблюдения

```
class Report {
  String getContent() { /* ... */ }
}

class ReportPrinter {
  void printReport(Report report) { /* ... */ }
}
```

Open/Closed Principle

Классы должны быть открыты для расширения, но закрыты для изменения.

Пример нарушения

```
class Logger {
  void log(LogType type, String message) {
    switch (type) {
      case CONSOLE -> logToConsole(message);
      case FILE -> logToFile(message);
    }
  }
}
```

Пример соблюдения

```
interface Logger {
  void log();
}

class ConsoleLogger implements Logger {
  void log(String message) { /* ... */ }
}

class FileLogger implements Logger {
  void log(String message) { /* ... */ }
}
```

Liskov Substitution Principle

Использование наследников вместо базового класса не должно ломать клиентский код.

Пример нарушения

```
class BankAccount {
  void withdraw(double amount) { /* ... */ }
}

class FixedDepositAccount extends BankAccount {
  void withdraw(double amount) {
    throw new UnsupportedOperationException();
  }
}
```

Пример соблюдения

```
class AccountException extends Exception {}

class BankAccount {
  void withdraw(double amount) throws AccountException { /* ... */ }
}

class FixedDepositAccount extends BankAccount {
  void withdraw(double amount) throws AccountException {
    throw new AccountException();
  }
}

class CreditCardAccount extends BankAccount {
  void withdraw(double amount) throws AccountException { /* ... */ }
}
```

Interface Segregation Principle

Много узких интерфейсов лучше, чем один широкий.

Пример нарушения

```
interface MultyFunctionPrinter {
  void print()
  void scan()
}

class RegularScanner implements
    MultyFunctionPrinter {
  void print() {
    throw new UnsupportedOperationException()
  }
  void scan() { /* ... */ }
}
```

Пример соблюдения

```
interface Printer {
  void print()
}

interface Scanner {
  void scan()
}

class RegularScanner implements Scanner {
  void scan() { /* ... */ }
}
```

Dependency Inversion Principle

Классы верхних уровней не должны зависеть от нижних. Зависимость от абстракции, а не от реализации.

Пример нарушения

```
class PgDatabase { /* ... */ }

class DataService {
  private PgDatabase database = new PgDatabase();
  /* ... */
}
```

Пример соблюдения

```
interface Database { /* ... */ }

class PgDatabase implements Database { /* ... */ }

class DataService {
  private Database database;
  public DataService(Database database) {
    this.database = database;
  }
  /* ... */
}
```

Coupling & Cohesion

Что это

Coupling — степень связанности между разными блоками кода.

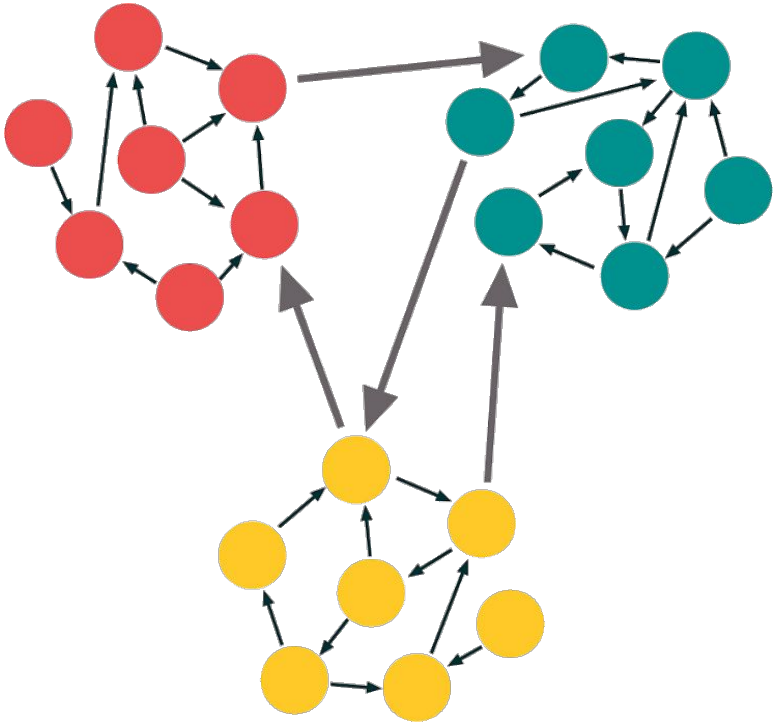
Cohesion — степень логической связанности кода внутри одного блока.

➕ Низкий coupling и высокий cohesion — хорошо.
➖ Высокий coupling и низкий cohesion — плохо.

Низкий coupling и высокий cohesion

Идеальный случай.

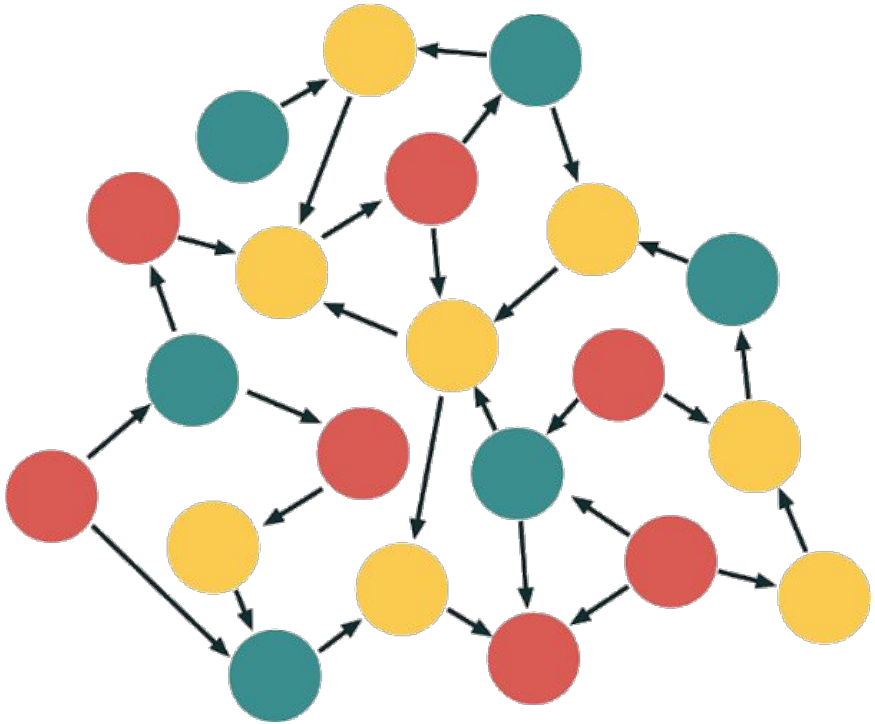
Связанный код объединен в блоки, которые не сильно зависят друг от друга.



God Object

Антипаттерн.

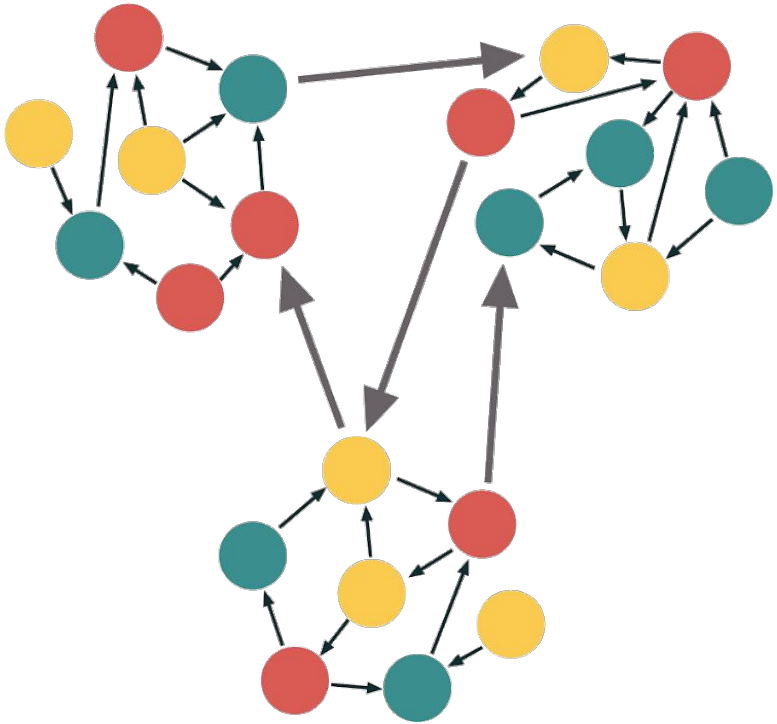
Один большой кусок кода, который выполняет всю работу сразу.



Низкий cohesion

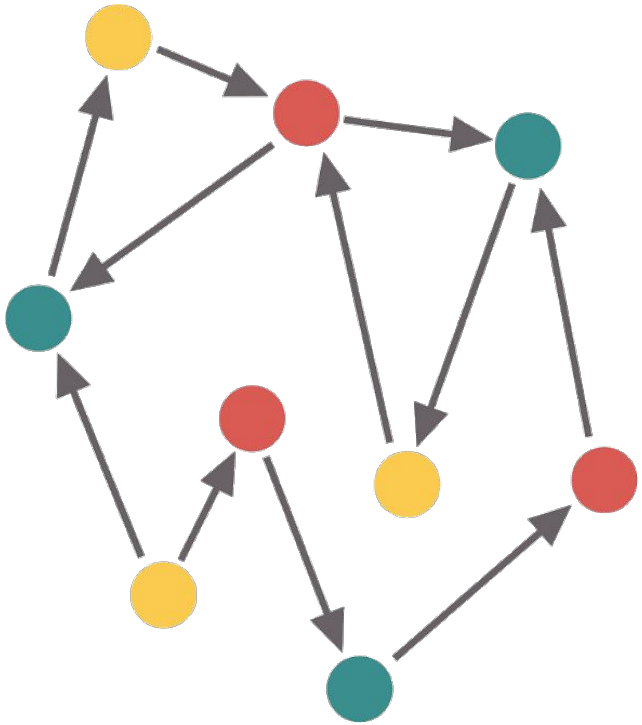
Код поделен на блоки, но границы выбраны неправильно — код внутри блоков логически плохо связан.

Обычно нарушает Single Responsibility Principle.



Destructive Decoupling

В попытке уменьшить coupling был потерян cohesion. В итоге, код насколько разделен, что сложно понять, для чего нужна каждая его часть.



Композиция vs. наследование

Наследование

Преимущества

- ➕ Легкое переиспользование кода.
- ➕ Позволяет использовать полиморфизм.

Недостатки

- ➖ Нельзя отказаться от интерфейса или реализации базового класса.
- ➖ Необходимо следить за тем, чтобы не сломать логику базового класса.
- ➖ Изменения в базовом классе могут сломать подклассы.
- ➖ Нарушение инкапсуляции базового класса.
- ➖ Может привести к разрастанию иерархии классов.

Наследование

Когда использовать

- ✔ Один класс — подтип другого. Связь можно описать словом «является».
- ✔ Подкласс должен содержать в себе всю логику базового с незначительными изменениями или дополнениями.
- ✔ Есть необходимость использовать подкласс вместо базового.

Композиция

Использование логики класса путем добавления его экземпляра в качестве поля.

```
class Strings {
  void tune() { /* ... */ }
}

class Guitar {
  Strings strings;

  void tuneStrings() {
    strings.tune();
  }
}
```

Композиция

Преимущества

- ➕ Гибкость и возможность менять классы во время выполнения.
- ➕ Классы меньше зависят друг от друга.
- ➕ Инкапсуляция логики используемого класса.
- ➕ Можно использовать только необходимую часть логики.

Недостатки

- ➖ Лишний код, если большинство методов просто вызывают методы используемого класса.
- ➖ Не всегда позволяет использовать полиморфизм.

Композиция

Когда использовать

- ✔ Отношение между классами можно описать словом «содержит».
- ✔ Нужна возможность менять логику во время выполнения.
- ✔ Использование только части логики исходного класса.
- ✔ Исходный класс закрыт для наследования.

Вывод

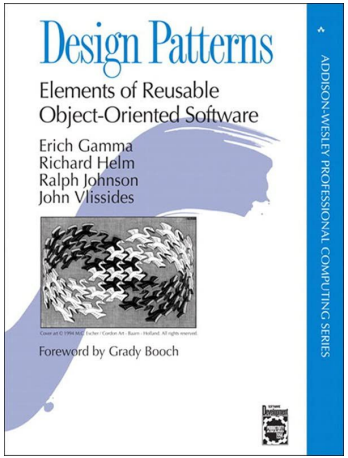
Нужно использовать то, что больше подходит в конкретной ситуации.

Паттерны проектирования

Что это

Паттерны проектирования — это типовые решения определенных задач при проектировании ПО.

Немного истории



Первая книга в 1994 г.:

Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides «Design Patterns: Elements of Reusable Object-Oriented Software»

она же «Book by the gang of four»

она же «GoF book»

Зачем

- Скорость разработки
- Эффективность
- Удобство коммуникации

Виды

- Порождающие
- Структурные
- Поведенческие

Порождающие паттерны

Описывают гибкое создание объектов без внесения в программу лишних зависимостей.

В рамках лекции:

- Builder

Остальные GoF:

- Singleton
- Abstract Factory
- Factory Method
- Prototype

Builder (Строитель)

Позволяет создавать сложные объекты пошагово. Дает возможность использовать один и тот же код создания для получения разных представлений объектов.

Объяснение на примере:
https://github.com/blackoutks/hh-school-2026-oop-patterns/tree/master/src/ru/hh/school/ooppatterns/creational/builder

Builder (Строитель)

Когда использовать

- ✔ Слишком много параметров в конструкторе.
- ✔ Нужна возможность создавать различные конфигурации объекта.
- ✔ Сложное конструирование объекта.

Builder (Строитель)

Преимущества

- ➕ Позволяет создавать сложные объекты пошагово.
- ➕ Позволяет использовать один и тот же код для создания различных продуктов.
- ➕ Изолирует сложный код сборки продукта от его основной бизнес-логики.

Недостатки

- ➖ Усложняет код программы из-за введения дополнительных классов.
- ➖ Клиент будет привязан к конкретным классам строителей.

Структурные паттерны

Описывают построение различных связей между объектами.

В рамках лекции:

- Adapter
- Proxy

Остальные GoF:

- Decorator
- Bridge
- Composite
- Facade
- Flyweight

Adapter (Адаптер)

Обеспечивает взаимодействие несовместимых интерфейсов.

Объяснение на примере:
https://github.com/blackoutks/hh-school-2026-oop-patterns/tree/master/src/ru/hh/school/ooppatterns/structural/adapter

Adapter (Адаптер)

Когда использовать

- ✔ Использование стороннего класса с неподходящим интерфейсом.
- ✔ Добавление общей функциональности к существующим подклассам, если нет возможности расширить базовый класс.

Adapter (Адаптер)

Преимущества

- ➕ Следует принципу единственной ответственности.
- ➕ Делает логику независимой от внешних интерфейсов.
- ➕ Позволяет сделать код совместимым со сторонними библиотеками или легаси-кодом.

Недостатки

- ➖ Может усложнить код, добавляя новые слои.
- ➖ Может скрывать плохой дизайн.

Proxy (Заместитель)

Заменяет реальные объекты, перехватывая вызовы и позволяя сделать что-то до или после них. Обычно нужен для контроля доступа к объектам.

Объяснение на примере:
https://github.com/blackoutks/hh-school-2026-oop-patterns/tree/master/src/ru/hh/school/ooppatterns/structural/proxy

Proxy (Заместитель)

Когда использовать

- ✔ Ленивая инициализация.
- ✔ Защита доступа.
- ✔ Работа с удаленным сервисом как с локальным.
- ✔ Логирование запросов.
- ✔ Кеширование объектов.

Proxy (Заместитель)

Преимущества

- ➕ Позволяет контролировать объект незаметно для клиента.
- ➕ Может работать, даже если реальный объект еще не создан.
- ➕ Может контролировать жизненный цикл объекта.
- ➕ Может улучшить производительность через отложенную инициализацию.

Недостатки

- ➖ Усложняет код дополнительными классами.
- ➖ Может ухудшить производительность.

Поведенческие паттерны

Описывают взаимодействие между объектами.

В рамках лекции:

- Strategy

Остальные GoF:

- Chain of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Template Method
- Visitor

Strategy (Стратегия)

Определяет семейство схожих алгоритмов, инкапсулирует каждый из них и делает их взаимозаменяемыми. Позволяет выбирать алгоритм поведения во время выполнения.

Объяснение на примере:
https://github.com/blackoutks/hh-school-2026-oop-patterns/tree/master/src/ru/hh/school/ooppatterns/behavioral/strategy

Strategy (Стратегия)

Когда использовать

- ✔ Использование разных вариаций одного алгоритма.
- ✔ Есть множество похожих классов, отличающихся только некоторым поведением.
- ✔ Хочется изолировать детали реализации, данные и зависимости от класса-клиента.
- ✔ Разные вариации сделаны в виде большого дерева условных операторов.

Strategy (Стратегия)

Преимущества

- ➕ Следует принципу открытости/закрытости.
- ➕ Горячая замена алгоритмов на лету.
- ➕ Простое добавление новых алгоритмов.
- ➕ Изолирует код и данные алгоритма.

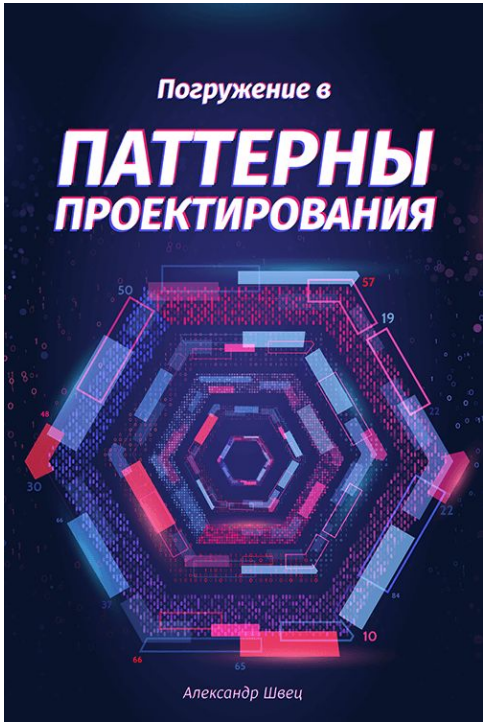
Недостатки

- ➖ Усложняет код дополнительными классами.
- ➖ Клиент должен знать разницу между стратегиями.

Ссылки

- Примеры из лекции: https://github.com/blackoutks/hh-school-2026-oop-patterns
- Хорошее объяснение всех паттернов GoF (недоступен из России): https://refactoring.guru/ru/design-patterns
- Доклад Евгения Борисова — Spring Patterns: https://youtu.be/61duchvKI6o

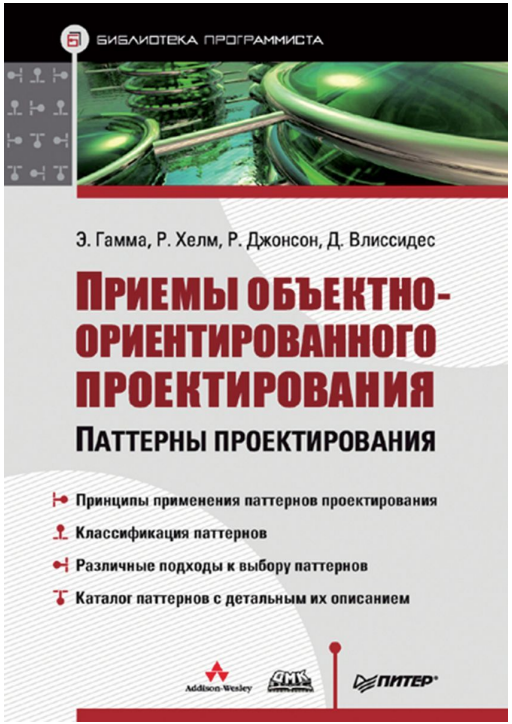
Что почитать



Александр Швец
«Погружение в паттерны проектирования»

Довольно понятно с примерами, схемами и псевдокодом описывает все паттерны GoF и некоторые принципы проектирования.
По сути — это refactoring.guru в виде книги.

Что почитать



Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес «Приемы объектно-ориентированного проектирования: Паттерны проектирования»

В оригинале:

Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides «Design Patterns: Elements of Reusable Object-Oriented Software»

Классика от GoF. Написана довольно академично.

Что почитать

Э. Фримен, Э. Робсон, К. Сьерра, Б. Бейтс «Head First. Паттерны проектирования»

В оригинале:
Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra «Head First Design Patterns»

Своеобразная веселая подача с диалогами и картинками.

В качестве справочника вряд ли подойдет.

Спасибо за внимание!